Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

ОТЧЁТ
по ознакомительной практике

Выполнил: Е. П. Рассохов

Студент группы
421703
Проверил: Н. В. Зотов

Минск 2025

СОДЕРЖАНИЕ

Введение 3
1 Формализация и сравнительный анализ декларативных языков программирования: LISP и LINQ 4
2 Формализация и сравнительный анализ инструментов для разработки графовых баз данных: Neo4j и Blazegraph 8
Заключение 13
Список использованных источников 14

ВВЕДЕНИЕ

Цель:

Провести сравнительный анализ современных технологий разработки интеллектуальных систем, закрепить навыки формализации информации, а также приобрести и развить навыки исследования, анализа и сравнения технологических решений.

Задачи:

1. Изучить декларативные языки порграммирования (LISP и LINQ) как инструменты формализации знаний, провести анализ их особенностей, выявить сходства и различия между ними..
2. Рассмотреть современные инструменты для работы с графовыми базами данных (Neo4j и Blazegraph), выявить их функциональные возможности и подходы к представлению знаний, сходства и отличительные особенности.
3. Формализовать полученные результаты в виде SCn-кода.
4. Оформить библиографические источники к полученным результатам по ГОСТ 7.1-2003.

1 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ДЕКЛАРАТИВНЫХ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ: LISP И LINQ

Lisp

:= [Язык программирования, разработанный Джоном Маккарти]

:= [Lisp (LISt Processing)]

∈ *язык программирования*

⇒ *разработчик**:

- *Джон Маккарти*

⇒ *год создания**:

1958

⇒ *основная парадигма**:

мультипарадигменный: функциональное, императивное, объектно-ориентированное, метапрограммирование

⇒ *описание**:

[Lisp — один из старейших языков программирования, основанный на обработке списков и принципе гомоиконичности. Программы на Lisp представляются в виде S-выражений, где код и данные имеют идентичное списковое представление, что облегчает динамическое преобразование и генерацию кода.]

⇒ *описание**:

[Язык широко используется в исследованиях искусственного интеллекта, разработке экспертных систем, обработке символьной информации, а также для создания доменных специфических языков (DSL).]

⇒ *назначение**:

[Lisp применяется для построения интеллектуальных систем, экспериментальных вычислительных платформ и учебных проектов, демонстрирующих идеи функционального и метапрограммирования.]

⇒ *основные конструкции**:

{
- [S-выражения (списки)]
- [определение функций с использованием `defun`]
- [условные конструкции: `if`, `cond`]
- [рекурсия как основной метод итерации]
- [макросы для создания новых синтаксических конструкций]
- [функции для работы со списками: `car`, `cdr`, `cons`]

}

⇒ *поддержка**:

- [динамическая типизация]
- [автоматическая сборка мусора]
- [поддержка объектов первого класса]

⇒ *применение**:

- *искусственный интеллект*
- *экспертные системы*
- *метапрограммирование и создание DSL*
- *обработка символьной информации*

⇒ *реализации**:

{
- *Common Lisp*
- *Scheme*

- *Clojure*
- *Racket*
- *Emacs Lisp*
- *AutoLISP*

}

⇒ *особенности**:

{
- [гомоиконичность (код как данные)]
- [мощная макросистема]
- [выражения в виде списков (S-выражения)]
- [поддержка функций первого класса и рекурсии]

}

⇒ *ограничения**:

{
- [необычный для новичков синтаксис (много скобок)]
- [иногда сниженная производительность без особых оптимизаций]

}

⇒ *библиографические источники**:

{
- [Steele G. L. Common Lisp the Language. – Prentice Hall, 1990.]
- [Clocksin W. F., Mellish C. S. Programming in LISP. – Springer, 2003.]
- [Lisp (programming language) – Wikipedia, https://en.wikipedia.org/wiki/Lisp_-(programming_language)]

}

LINQ

:= [Язык интегрированных запросов в рамках платформы .NET, разработанный Microsoft]

:= [LINQ (Language Integrated Query)]

∈ *интегрированный язык запросов*

⇒ *разработчик**:

- *Microsoft*
- *Anders Hejlsberg & др.*

⇒ *год создания**:

2007

⇒ *основная парадигма**:

декларативное программирование, функциональный стиль

⇒ *описание**:

[LINQ — компонент платформы .NET, позволяющий выполнять запросы к различным источникам данных (коллекциям, XML, базам данных, DataSet и пр.) с использованием единого синтаксиса, встроенного в языки, такие как C# и VB.NET.]

⇒ *описание**:

[LINQ объединяет декларативный стиль запросов с функциональными операциями, такими как фильтрация, проекция, группировка и объединение данных, что упрощает обработку и трансформацию информации.]

⇒ *назначение**:

[LINQ применяется для создания лаконичного и читаемого кода при работе с разнородными источниками данных, обеспечивая типобезопасные запросы и интеграцию с .NET-окружением.]

⇒ *основные конструкции**:

{
- [запрос с использованием ключевых слов: `from`, `where`, `select`, `group`]
- [метод-синтаксис через методы расширения (например, `Where`, `Select`, `GroupBy`)]

- [выражения лямбда-функций]

}

⇒ *поддержка**:

- [LINQ to Objects]
- [LINQ to SQL]
- [LINQ to XML]
- [LINQ to DataSet]
- [PLINQ (параллельный LINQ)]

⇒ *применение**:

- *разработка бизнес-приложений на платформе .NET*
- *анализ и преобразование данных*
- *интеграция разнородных источников данных*
- *статический анализ данных и оптимизация запросов*

⇒ *реализации**:

{
- *Microsoft .NET Framework*
- *Visual Studio*

}

⇒ *особенности**:

{
- [унифицированный синтаксис для доступа к различным источникам данных]
- [интеграция с языками .NET (C#, VB.NET)]
- [поддержка функциональных операторов]
- [типобезопасность запросов]
- [ленивые вычисления и оптимизация выполнения запросов]

}

⇒ *ограничения**:

{
- [зависимость от платформы .NET]
- [сложность для тех, кто не знаком с декларативным стилем программирования]

}

⇒ *библиографические источники**:

{
- [LINQ overview - .NET | Microsoft Learn, https://learn.microsoft.com/en-us/dotnet/standard/linq/]
- [LINQ - Overview - Online Tutorials Library, https://www.tutorialspoint.com/linq/linq$_overview.htm$]
- [LINQ Tutorial For Beginners and Professionals - Dot Net Tutorials, https://dotnettutorials.net/course/linq/]

}

Сравнение языков программирования Lisp и LINQ

= ⟨
- *Lisp*
- *LINQ*

⟩

⇒ *сходства**:

{
- [Оба предлагают декларативное описание операций над данными]
- [Поддерживают элементы функционального программирования]
- [Позволяют лаконично выражать преобразования данных]

}

⇒ *различия**:

{
- [Lisp — полнофункциональный язык программирования с мощной макросистемой, а LINQ — встроенный компонент для запросов в .NET]

- [Lisp ориентирован на метапрограммирование и обработку списков, LINQ предназначен для унифицированного доступа к разнообразным источникам данных]
- [Основное применение Lisp — в ИИ и обработке символьной информации, LINQ используется преимущественно в бизнес-приложениях и интеграции данных]

}

⇒ *рекомендации по выбору**:

{
- [Используйте Lisp для проектов, требующих гибкости метапрограммирования, экспериментов с синтаксисом и исследований в области ИИ]
- [Используйте LINQ для унифицированного и типобезопасного доступа к данным в приложениях на платформе .NET]

}

2 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ИНСТРУМЕНТОВ ДЛЯ РАЗРАБОТКИ ГРАФОВЫХ БАЗ ДАННЫХ: NEO4J И BLAZEGRAPH

Neo4j

:= [Графовая база данных, разработанная компанией Neo4j, Inc.]

:= [Система управления графовыми данными]

∈ *система управления базами данных*

⇒ *разработчик**:

Neo4j, Inc.

⇒ *год создания**:

2007

⇒ *текущая стабильная версия**:

[4.4.12]

⇒ *дата выпуска текущей версии**:

[15 декабря 2024]

⇒ *лицензия**:

GPL v3 (Community Edition), Коммерческая лицензия (Enterprise Edition)

⇒ *язык разработки**:

Java

⇒ *назначение**:

[Хранение, анализ и визуализация графовых данных. Система предназначена для эффективного моделирования взаимосвязанных данных, поиска путей, анализа социальных сетей, рекомендательных систем и других задач, где критична обработка связей между объектами.]

⇒ *варианты исполнения**:

{• *Neo4j Community Edition*

:= [Открытая версия, предоставляющая базовый функционал для разработки и тестирования графовых приложений.]

⇒ *примечание**:

[Бесплатно распространяется под лицензией GPL v3.]

• *Neo4j Enterprise Edition*

:= [Расширенная коммерческая версия с дополнительными возможностями по масштабированию, безопасности и управлению.]

• *Neo4j Aura*

:= [Облачное решение для развертывания Neo4j в виде полностью управляемого сервиса.]

}

⇒ *функциональные возможности**:

{• [Нативное хранение данных в виде узлов, связей и свойств]

⇒ *пример использования**:

[Моделирование социальных сетей, ИТ-инфраструктуры или биологических систем.]

• [Декларативный язык запросов Cypher]

⇒ *пример использования**:

[Определение сложных паттернов поиска по графу без необходимости выполнения сложных соединений (JOIN).]

•

[Поддержка ACID-транзакций]
- [Встроенные графовые алгоритмы и процедуры (включая APOC)]
- [Интеграция с различными API (Java, Python, .NET, REST, Bolt)]

}

⇒ *поддерживаемые языки запросов и интерфейсы**:

{
- *Cypher*
- *Bolt (протокол)*
- *REST API*

}

⇒ *экосистема расширений**:

{
- *APOC Procedures*
  - ⇒ *назначение**:
    [Набор вспомогательных процедур и функций для расширенной обработки графовых данных.]
- *Graph Data Science Library*
  - ⇒ *назначение**:
    [Библиотека для выполнения графовых алгоритмов и анализа данных.]
- *Neo4j Bloom*
  - ⇒ *назначение**:
    [Интерактивное средство визуализации и анализа графов.]

}

⇒ *преимущества**:

{
- [Высокая производительность при выполнении сложных запросов по множеству взаимосвязей]
- [Гибкая, схема не требующая предопределения структура данных]
- [Интуитивный и декларативный синтаксис языка запросов Cypher]
- [Нативное хранение и обработка графовых данных обеспечивает эффективные обходы и анализ связей]
- [ACID-транзакции гарантируют целостность данных]

}

⇒ *недостатки**:

{
- [Ограничения масштабируемости Community Edition для критичных промышленных задач]
- [Высокие требования к ресурсам при работе с очень большими графами в Enterprise-версии]
- [Зависимость от JVM может влиять на конфигурацию и производительность в некоторых сценариях]

}

⇒ *библиографические источники**:

{
- [Neo4j Documentation [Электронный ресурс] // https://neo4j.com/docs/ (дата обращения: 12.05.2025).]
- [What is a Graph Database? – Neo4j, https://neo4j.com/developer/graph-database/ (дата обращения: 12.05.2025).]
- [Robinson I., Webber J., Eifrem E. Graph Databases: New Opportunities for Connected Data. – O'Reilly, 2015.]

}

Blazegraph

:= [Графовая база данных высокой производительности, разработанная компанией Blazegraph Inc.]

:= [Система управления графовыми данными]

∈ *система управления базами данных*

⇒ *разработчик**:
Blazegraph Inc.

⇒ *год создания**:
2007

⇒ *текущая стабильная версия**:
[2.1.6]

⇒ *дата выпуска текущей версии**:
[10 декабря 2024]

⇒ *лицензия**:
GPL v2 (Community Edition), коммерческая лицензия (Enterprise Edition)

⇒ *язык разработки**:
Java

⇒ *назначение**:
[Система предназначена для хранения, анализа и визуализации графовых данных. Blazegraph позволяет моделировать и эффективно обрабатывать взаимосвязанные данные в задачах социальных сетей, рекомендательных систем, биоинформатики и других областях, где важна обработка сложных взаимосвязей между объектами.]

⇒ *варианты исполнения**:
{
- *Blazegraph Community Edition*
  ⇒ *описание**:
  [Открытая версия с полным функционалом для разработки и тестирования графовых приложений, распространяемая бесплатно под лицензией GPL v2.]
- *Blazegraph Enterprise Edition*
  ⇒ *описание**:
  [Расширенная версия с дополнительными возможностями масштабирования, безопасности и поддержки, предназначенная для критически важных корпоративных приложений.]
- *Blazegraph Cloud Edition*
  ⇒ *описание**:
  [Облачное решение, предоставляющее управляемый сервис для быстрого развертывания и эксплуатации графовой базы данных.]

}

⇒ *функциональные возможности**:
{
- [Нативное хранение графовых данных]
  ⇒ *уточнение**:
  [Представление данных в виде узлов, ребер и их свойств, что облегчает моделирование комплексных взаимосвязей.]
- [Поддержка декларативного языка запросов SPARQL]
  ⇒ *уточнение**:
  [Выполнение сложных запросов с использованием стандарта SPARQL для поиска паттернов в графе.]
- [Интеграция с Blueprints API]
-

[Масштабирование и высокая производительность]

⇒ *уточнение**:

[Обработка до десятков миллиардов ребер на одной машине, что позволяет использовать систему в условиях промышленных нагрузок.]

- [Встроенные алгоритмы графового анализа]

  ⇒ *уточнение**:

  [Поддержка алгоритмов для поиска кратчайших путей, кластеризации, анализа социальных сетей и других задач графового анализа.]

- [Интеграция с внешними источниками данных через HTTP/REST]

}

⇒ *поддерживаемые языки и протоколы**:

{
- *SPARQL*
- *Blueprints API*
- *HTTP/REST API*

}

⇒ *экосистема расширений**:

{
- *Blazegraph Workbench*

  ⇒ *назначение**:

  [Визуальный интерфейс для управления базой данных, выполнения запросов и анализа графовых данных.]

- *Интеграция с Apache Tinkerpop*

  ⇒ *назначение**:

  [Возможность взаимодействия с экосистемой Tinkerpop для расширенных графовых вычислений и обмена данными.]

- *Плагины и процедуры для расширения возможностей SPARQL*

}

⇒ *преимущества**:

{
- [Высокая производительность при выполнении сложных графовых запросов]
- [Нативное хранение данных обеспечивает эффективный обход и анализ отношений]
- [Масштабируемость для крупных промышленных приложений]
- [Гибкость интеграции через поддержку различных API и протоколов]

}

⇒ *недостатки**:

{
- [Ограниченная документация по коммерческим функциям для начинающих пользователей]
- [Сложности настройки кластерных решений в Enterprise-версии]

}

⇒ *библиографические источники**:

{
- [Blazegraph Documentation [Электронный ресурс] // https://blazegraph.com/docs/ (дата обращения: 30.05.2025).]
- [What is Blazegraph? – Blazegraph, https://blazegraph.com/ (дата обращения: 30.05.2025).]
- [Robinson I., Webber J., Eifrem E. Graph Databases: New Opportunities for Connected Data. – O'Reilly, 2015.]

}

Сравнение систем управления графовыми данными Neo4j и Blazegraph

= ⟨ • *Neo4j*
• *Blazegraph*
⟩

⇒ *сходства**:
{ • [Обе системы предназначены для управления графовыми данными]
• [Используют графовую модель данных, где основными элементами являются узлы, ребра и их свойства]
• [Поддерживают ACID-транзакции для обеспечения целостности и надежности данных]
• [Применяются в задачах анализа взаимосвязей (социальные сети, рекомендательные системы, биоинформатика и др.)]
• [Обеспечивают интеграцию через API (Java API, HTTP/REST, протоколы доступа)]
• [Обе системы имеют открытые версии и варианты для корпоративного использования]
• [Поддерживают визуализацию графовых данных посредством специализированных клиентских приложений или панелей управления]
}

⇒ *различия**:
{ • [Neo4j использует модель *labeled property graph* и язык запросов Cypher, в то время как Blazegraph ориентирован на RDF-данные и реализует SPARQL в качестве основного языка запросов]
• [Neo4j обладает обширной экосистемой, активным сообществом и множеством интеграционных инструментов (например, Neo4j Bloom для визуализации), а Blazegraph известен своей высокой масштабируемостью для распределённого хранения RDF-данных]
• [Blazegraph оптимизирован для дедуктивных RDF-моделей и сложных SPARQL-запросов, в то время как Neo4j предоставляет более удобный синтаксис и инструменты для работы с графами в бизнес-приложениях]
• [В части лицензирования: Neo4j доступен как в Community Edition (GPLv3) с коммерческими вариантами, так и в полной корпоративной редакции, а Blazegraph распространяется под лицензией AGPL с опциями для коммерческого использования]
}

⇒ *рекомендации по выбору**:
{ • [Neo4j рекомендуется для проектов, где важны удобство разработки, богатая экосистема, активное сообщество и интуитивный язык запросов Cypher]
• [Blazegraph целесообразен для задач, требующих высокой производительности при обработке огромных объемов RDF-данных, реализации дедуктивных моделей и масштабируемых распределенных систем в рамках Semantic Web]
• [Для корпоративных проектов, где требуется комплексная интеграция с современными веб-технологиями и мощные средства визуализации графов, Neo4j может предоставить более широкий функционал]
• [Если же проект ориентирован на обработку данных по стандартам RDF с акцентом на масштабирование и высокопроизводительные SPARQL-запросы, Blazegraph будет оптимальным выбором]
}

ЗАКЛЮЧЕНИЕ

В ходе проведённого исследования был осуществлён сравнительный анализ современных технологий разработки интеллектуальных систем: языков логического программирования и инструментов онтологического моделирования.

В первом разделе работы были подробно рассмотрены гибридные языки программирования LISP и LINQ. Проанализированы их синтаксические и семантические особенности, выявлены общие черты и различия в подходах к представлению и обработке знаний. Результаты анализа были формализованы с помощью SCn-кода.

Во втором разделе исследованы современные инструменты для работы с графовыми базами данных Neo4j и Blazegraph. Проведен анализ их функциональных возможностей, методов представления знаний, а также особенностей архитектуры. Информация о каждом инструменте также была формализована на SCn-коде.

Результаты работы могут быть использованы при проектировании гибридных интеллектуальных систем, сочетающих возможности логического вывода и онтологического моделирования, а также при выборе и обосновании технологических решений для конкретных прикладных задач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Steele, G. L. Common Lisp the Language / G. L. Steele. — Prentice Hall, 1990.

[2] Clocksin, W. F. Programming in LISP / W. F. Clocksin, C. S. Mellish. — Springer, 2003.

[3] Lisp (programming language). — Электронный ресурс. Дата обращения: 30.05.2025. https://en.wikipedia.org/wiki/Lisp_(programming_language).

[4] Linq overview - .net | microsoft learn. — Электронный ресурс. Дата обращения: 30.05.2025. https://learn.microsoft.com/en-us/dotnet/standard/linq/.

[5] Linq - overview - online tutorials library. — Электронный ресурс. Дата обращения: 30.05.2025. https://www.tutorialspoint.com/linq/linq_overview.htm.

[6] Linq tutorial for beginners and professionals - dot net tutorials. — Электронный ресурс. Дата обращения: 30.05.2025. https://dotnettutorials.net/course/linq/.

[7] Blazegraph documentation [Электронный ресурс]. — Электронный ресурс. Дата обращения: 30.05.2025. https://blazegraph.com/docs/.

[8] What is blazegraph? – blazegraph. — Электронный ресурс. Дата обращения: 30.05.2025. https://blazegraph.com/.

[9] Robinson, I. Graph Databases: New Opportunities for Connected Data / I. Robinson, J. Webber, E. Eifrem. — O'Reilly, 2015.

[10] Microsoft. linq (language integrated query) [Электронный ресурс]. — Электронный ресурс. Дата обращения: 30.05.2025. https://docs.microsoft.com/ru-ru/dotnet/csharp/programming-guide/concepts/linq/.